

Newton and Gauss-Newton Methods for GPS Positioning

Alexandru Tudor

May 7, 2026

1 Abstract

The modern Global Positioning System (GPS) works by approximating the solution to a complex non-linear system of equations in the presence of atmospheric noise. This project aims to utilize two numerical methods to approximate a GPS receiver's position: the Multivariate Newton Method applied to a system of 4 satellites, and the Gauss-Newton method applied to an overdetermined system of 10 satellites. Both of these methods were written as programs in Python, and tested on real satellite data. The Newton method converged quadratically in 7 iterations to an exact solution of the system. The Gauss-Newton method converged super-linearly in early iterations and linearly in later iterations in 11 total iterations to a tolerance of $\|\Delta u_k\|_2 < 10^{-8}$. Although the Newton method appears to converge perfectly, the Gauss-Newton method achieves a more geographically accurate estimate in position due to the atmospheric noise present in the data.

2 Introduction & Mathematical Background

2.1 The Problem

The GPS determines the position of a receiver by measuring the time it takes for signals to reach satellites. This time is used to compute the distance between the receiver and a satellite, which is also known as a pseudorange. The process of trilateration requires a minimum of 3 satellites to identify the exact x , y , and z coordinates of a receiver. In this process, each satellite defines a sphere of possible locations, and with 3 satellites, the intersection of the spheres identifies the receiver in a 3-dimensional space. However, since a receiver's internal clock is not synchronized with the atomic clocks on the satellites, clock bias, d , is introduced. Thus, since there are 4 total unknowns, a minimum of 4 satellites are required to determine the position of a receiver - 3 responsible for trilateration, and 1 responsible for clock bias.

However, in practice, satellite signals are corrupted by atmospheric noise, thus modern GPS systems utilize more than 4 total satellites to identify the position of a receiver. This however, leads to an overdetermined system that cannot be simply solved. Instead, this problem is solved by minimizing the sum of squared residuals to produce an accurate position estimate.

2.2 Mathematical Formulation

Let the unknown state vector be $u = [x, y, z, d]^T$. For each visible satellite i with known position (x_i, y_i, z_i) and measured pseudorange P_i , we obtain the following residual function:

$$f_i(x, y, z, d) = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + c \cdot d - P_i$$

where $c = 299,792,458$ m/s is the speed of light. The residual function f_i measures the magnitude in difference between the predicted pseudorange from the current guess and the actual pseudorange measured

from satellite i . We then define the full system $F : R^4 \rightarrow R^N$ as the vector-valued function of all N residuals:

$$F(u) = \begin{bmatrix} f_1(u) \\ f_2(u) \\ \vdots \\ f_N(u) \end{bmatrix}$$

Thus, in a hypothetical noise-free 4-satellite case ($N = 4$), we aim to find a u^* such that $F(u^*) = 0$. In a more realistic noisy overdetermined case ($N > 4$), we may not be able to solve for $F(u) = 0$, so we instead minimize:

$$\min_u \|F(u)\|_2^2 = \min_u \sum_{i=1}^N f_i(u)^2$$

2.3 The Algorithms

2.3.1 Multivariate Newton Method

In the 4-satellite scenario ($N = 4$), we have a 4×4 system which has an exact solution. For each observation i , the algorithm first computes the straight line distance (R_i) from the most updated position guess (x, y, z) , to the position of the i -th satellite using:

$$R_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2}$$

Then, the program uses the computed value of R_i for each satellite, and computes the residual f_i for each satellite using:

$$f_i(u_k) = f_i(x, y, z, d) = R_i + c \cdot d - P_i$$

Note: x_i, y_i, z_i, P_i are all givens from the i -th satellite, and x, y, z, d are pulled from the most updated guess. Then, we compile the 4 calculated residuals into the vector-valued function $F(u_k)$:

$$F(u_k) = \begin{bmatrix} f_1(u_k) \\ f_2(u_k) \\ f_3(u_k) \\ f_4(u_k) \end{bmatrix}$$

The algorithm also generates the 4×4 Jacobian matrix, $J(u_k)$, for each iteration k using the following formulas to generate the i -th row:

$$J_{i,1} = \frac{x - x_i}{R_i}, \quad J_{i,2} = \frac{y - y_i}{R_i}, \quad J_{i,3} = \frac{z - z_i}{R_i}, \quad J_{i,4} = c$$

After obtaining $F(u)$ and $J(u_k)$, the program solves the linear system:

$$J(u_k)\Delta u_k = -F(u_k)$$

Then, the program updates the guess using:

$$u_{k+1} = u_k + \Delta u_k$$

After each iteration, the program stores the 2-norm of the update step $\|\Delta u_k\|_2$ as a measure of convergence. The algorithm then repeats the process with u_{k+1} until our iteration converges, or the 2-norm of the update step reaches the stopping criteria, which will be defined in a later section.

2.3.2 Gauss-Newton Method

In the scenario of more than 4 satellites ($N > 4$), the system is overdetermined and an exact solution utilizing the multivariate Newton method may not exist. Instead, we use the Gauss-Newton method, which minimizes the sum of squared residuals:

$$\min_u \|F(u)\|_2^2 = \min_u \sum_{i=1}^N f_i(u)^2$$

The Gauss-Newton algorithm similarly computes the $N \times 4$ Jacobian matrix using the formulas for the i -th row defined above, and compiles the N residuals into the vector-valued function $F(u_k)$:

$$F(u_k) = \begin{bmatrix} f_1(u_k) \\ f_2(u_k) \\ \vdots \\ f_N(u_k) \end{bmatrix}$$

Then, the Gauss-Newton method solves the minimization problem through solving the Normal Equations for each iteration k :

$$J(u_k)^T J(u_k) \Delta u_k = -J(u_k)^T F(u_k)$$

Then, the program updates the guess using:

$$u_{k+1} = u_k + \Delta u_k$$

After each iteration, the program stores the 2-norm of the update step $\|\Delta u_k\|_2$ as a measure of convergence. We continue this process until our iteration converges, or the 2-norm of the update step reaches the stopping criteria, which will be defined in a later section.

Note: For the Gauss-Newton method, the program will utilize $N = 10$ total satellites, resulting in an initial 10×4 system.

3 Implementation & Setup

3.1 Initialization

Both the Newton and Gauss-Newton solvers were initialized with the initial guess:

$$u_0 = [0.0, 0.0, 6,371,000.0, 0.0]^T$$

Here, the first 3 components represent the Earth's x , y , and z coordinates in meters, and the fourth component represents the clock bias d . The coordinate z was initialized to the value of the mean radius of Earth, which is about 6,371 km. This causes the initial guess to be placed on the Earth's surface, which is a reasonable starting point since GPS receivers are located near the Earth's surface. This ensures that our starting point is in the same region as the true solution, which is important for the local convergence of the Newton method.

3.2 Stopping Criteria

Both solvers have the stopping criteria:

$$\|\Delta u_k\|_2 < 10^{-8}$$

Furthermore, a maximum iteration limit of 50 was imposed to prevent an infinite loop in the event of a failure to converge.

3.3 Linear Algebra

In the case of the Newton method, we need to solve 4×4 linear system:

$$J(u_k)\Delta u_k = -F(u_k)$$

Also, for the Gauss-Newton method, we need to solve the overdetermined 10×4 linear system, which requires solving the Normal Equations:

$$J(u_k)^T J(u_k)\Delta u_k = -J(u_k)^T F(u_k)$$

In the algorithm, the direct Newton system and the Normal Equations from the Gauss-Newton method were solved utilizing the `numpy.linalg.solve` function in Python.

4 Results & Visualizations

4.1 Final Coordinates

Final Computed Position and Clock Bias				
Method	x (m)	y (m)	z (m)	d (s)
Newton	6,408,313.53	15,287.65	33,071.51	0.00006045
Gauss-Newton	3,394,721.21	-999,452.64	-1,313,550.65	-0.00963650

Table 1: Final computed position (x, y, z) and clock bias d for the Newton method (4 satellites) and Gauss-Newton method (10 satellites).

4.2 Convergence Data

Step Sizes and EOC				
k	$\ \Delta u_k\ _2$ (Newton)	EOC (Newton)	$\ \Delta u_k\ _2$ (Gauss-Newton)	EOC (Gauss-Newton)
1	9.081879×10^6	N/A	7.677611×10^6	N/A
2	7.314002×10^5	N/A	7.812135×10^5	N/A
3	2.067387×10^4	1.4156	1.269514×10^4	1.8027
4	1.748042×10^1	1.9841	5.973426×10^1	1.3009
5	1.245034×10^{-5}	2.0005	1.282542×10^{-1}	1.1464
6	1.717627×10^{-7}	0.3026	5.448297×10^{-4}	0.8889
7	0.000000×10^0	N/A	1.830955×10^{-6}	1.0429
8	—	—	2.292789×10^{-8}	0.7691
9	—	—	1.685071×10^{-8}	0.0703
10	—	—	3.084079×10^{-8}	-1.9627
11	—	—	9.463855×10^{-9}	-1.9545

Table 2: Step sizes $\|\Delta u_k\|_2$ and EOC per iteration k for both the Newton method (4 satellites) and Gauss-Newton method (10 satellites). Dashes represent no further iterations for a method due to full convergence.

4.3 Log-Error Plot

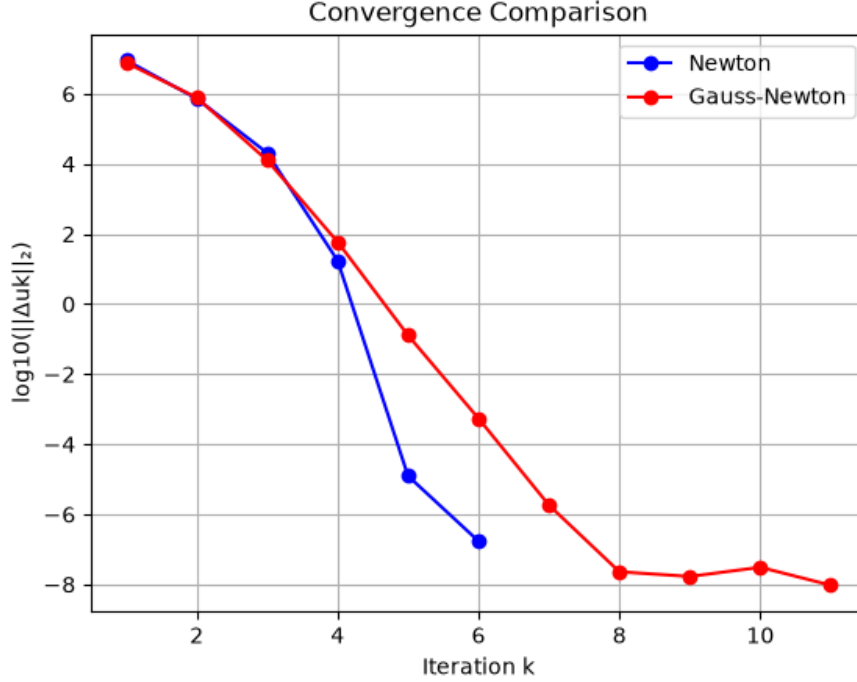


Figure 1: Log-error plot comparing the convergence of the Newton method and the Gauss-Newton method on each iteration k .

5 Discussion & Analysis

5.1 The “Perfect” Illusion

When performing the Newton method using $N = 4$ satellites, the program found a u^* such that $F(u^*) = 0$. This seems like a mathematically “perfect” solution, however this does not actually translate to geographical accuracy. The 4 initial satellite measurements provided were not perfect themselves, but rather they were impacted by atmospheric noise. Thus, the Newton method actually found the “perfect” solution to an inaccurate system.

On the contrary, the Gauss-Newton method used all $N = 10$ satellites, and although it didn’t generate a “perfect” solution, it minimized the sum of squared residuals, $\|F(u)\|_2^2$ across all measurements. By increasing N , the Gauss-Newton method did a better job at averaging out the noise of individual measurements, and generating a more accurate position estimate. This is consistent with the fact that modern day GPS uses as many satellites as possible to generate an accurate position.

5.2 Convergence Behavior

Analyzing Table 2, the Newton method has EOC values of 1.42, 1.98, and 2.00 at iterations 3, 4, and 5 respectively. This confirms that the Newton method achieved quadratic convergence, which was to be expected. The EOC value of 0.30 at iteration 6 is explained by the fact that the method has already essentially converged with a step size of 1.72×10^{-7} . The remaining step was simply dominated by floating point rounding errors, explaining the small EOC value.

Also analyzing Table 2, the Gauss-Newton method exhibited early EOC values of 1.80, 1.30, and 1.15 for iterations 3, 4, and 5 respectively. This shows superlinear order of convergence but not quadratic. After iteration 5, the EOC values are around 1 showing linear convergence. This is expected behavior for the Gauss-Newton method in the given scenario because the overdetermined system is noisy and has no true zero residual, thus it cannot achieve quadratic convergence. In the final iterations of the Gauss-Newton method, there were EOC values of 0.07, -1.96 , and -1.95 for iterations 9, 10, and 11 respectively. The negative EOC's indicate that the method has achieved the smallest error possible, and the only thing affecting the iterative steps is simply the noisy atmospheric data. This causes the iterations to oscillate, rather than decrease, which explains the near 0 and negative EOC values. This signifies that the algorithm has converged, and it is important to note that the negative EOC values carry no significance regarding the order of convergence for the Gauss-Newton method.

5.3 Algorithmic Limitations

If two of the chosen satellites were in the exact same position in the sky, it would cause two rows of the Jacobian matrix $J(u_k)$ to be identical. This is because the entries of the Jacobian matrix depend on (x_i, y_i, z_i) , which represent the coordinates of the i -th satellite. Thus, it would follow that $J(u_k)$ becomes rank deficient (no longer has full rank).

In the case of the Newton method, the 4×4 matrix $J(u_k)$ becomes singular, and thus the linear system $J(u_k)\Delta u_k = -F(u_k)$ has no unique solution. Hence, the algorithm would fail in this scenario.

In the case of the Gauss-Newton method, it holds that 4×4 matrix $J(u_k)^T J(u_k)$ also becomes singular, and thus the Normal Equations will also provide no solution.

Thus, one must be very careful to make sure that no two satellites used share the same position in the sky, or else it would cause the algorithms to fail.

6 References

1. Burden, Richard L., et al. *Numerical Analysis*. Cengage Learning, 2016.
2. Harris, Charles R., et al. "Array Programming with NumPy." *Nature*, vol. 585, no. 7825, 16 Sept. 2020, pp. 357–362, www.nature.com/articles/s41586-020-2649-2, <https://doi.org/10.1038/s41586-020-2649-2>.
3. Kaplan, Elliott D., and C. Hegarty. *Understanding GPS/GNSS: Principles and Applications*. Artech House, 2017.
4. Morichi, Luca, et al. "Pseudorange and Doppler-Based Positioning: Enabling Convergence of Least-Squares Estimation from MEO to LEO." *NAVIGATION: Journal of the Institute of Navigation*, vol. 73, no. 1, 5 Jan. 2026, p. navi.734, <https://doi.org/10.33012/navi.734>. Accessed 1 Apr. 2026.